

Prompt Engineering

Mini-Assessment — Solution

Name	B.Sai Ganesh
Register No	99230040272
Module	Prompt Engineering
Assessment	Mini Assessment: Multi-Step Prompt Workflow
Course	ICL GEN AI Mastery

Assessment Task

Design a 3-step workflow in which Step 1 uses few-shot prompting to extract key entities from a support ticket, Step 2 uses CoT prompting to determine an urgency level (low/medium/high) from those entities, and Step 3 uses JSON mode to output an object containing "entities", "urgency", and "summary".

Sample Support Ticket

"I can't log into my account after resetting my password. I have an important client meeting in 30 minutes and need access urgently. The password reset email arrived, but the new password isn't working."

Step 1 — Few-Shot Entity Extraction

Technique used: Few-shot prompting. Three examples anchor the desired entity extraction pattern and output format.

Prompt:

You are a support-ticket entity extraction assistant.

Extract the key entities from the support ticket.
Return only a bullet list.

Examples:

Ticket: "My credit card was charged twice for my subscription."

Entities:

- billing
- duplicate charge
- subscription

Ticket: "I cannot access my account because the login page keeps rejecting my password."

Entities:

- account
- login
- password
- access issue

Ticket: "The mobile application crashes whenever I try to upload a photo."

Entities:

- technical
- mobile application
- crash
- photo upload

Now extract the key entities from this ticket:

"I can't log into my account after resetting my password. I have an important client meeting in 30 minutes and need access urgently. The password reset email arrived, but the new password isn't working."

Entities:

Intermediate Output:

- account
- login
- password reset
- access issue
- urgent
- client meeting

Step 2 — CoT Urgency Reasoning

Technique used: Chain-of-thought prompting. Step 2 genuinely conditions its decision on the actual output produced by Step 1.

Prompt:

You are a senior customer-support triage specialist.

Determine the urgency level of the support ticket as LOW, MEDIUM, or HIGH.

Reason step by step using the extracted entities below.

Extracted entities:

{STEP_1_OUTPUT}

Urgency guidelines:

- LOW: Minor issue with little immediate impact.
- MEDIUM: Important issue but the user can reasonably wait.
- HIGH: Critical access/problem with an immediate deadline or major business impact.

Explain your reasoning briefly, then give the final urgency level.

Intermediate Output:

The user cannot access their account. The password reset has not resolved the login problem. There is an important client meeting in 30 minutes. Therefore, the issue has an immediate deadline and significant impact.

Urgency: HIGH

Step 3 — JSON Formatting

Technique used: JSON mode / structured output. The previous two outputs are passed into this step, which produces a valid JSON object with the required keys.

Prompt:

You are a support-ticket response formatter.

Using the extracted entities and urgency analysis below, return ONLY valid JSON.

Extracted entities:

{STEP_1_OUTPUT}

Urgency analysis:

{STEP_2_OUTPUT}

Return exactly this schema:

```
{
  "entities": [],
  "urgency": "low|medium|high",
  "summary": ""
}
```

Rules:

- "entities" must be an array of strings.
- "urgency" must contain only low, medium, or high.
- "summary" must be one concise sentence.
- Do not include markdown.
- Do not include any text outside the JSON object.

Final Output:

```
{
  "entities": [
    "account",
    "login",
    "password reset",
    "access issue",
    "urgent",
    "client meeting"
  ],
  "urgency": "high",
  "summary": "The customer cannot access their account after a failed password reset and needs urgent access before an important client meeting."
}
```

End-to-End Workflow

Raw Support Ticket

↓

Step 1: Few-shot prompt → Extract entities

↓

Step 2: CoT prompt + Step 1 output → Determine urgency

↓

Step 3: JSON mode + Step 1 and Step 2 outputs → Final structured JSON

The final JSON contains all three required keys: entities, urgency, and summary.

Assessment Checklist

Step 1 uses few-shot examples for entity extraction.

Step 2 uses CoT reasoning to determine low/medium/high urgency.

Step 2 uses the actual output from Step 1.

Step 3 uses structured JSON output.

The final JSON contains entities, urgency, and summary.

The final JSON is valid and can be parsed using `json.loads`.

The workflow includes an end-to-end example with intermediate outputs.

Conclusion

This workflow follows the Extract → Reason → Format design pattern from the assessment. Each stage uses the prompting technique best suited to its task, while the outputs are chained so that the later stages depend on the results of the earlier stages.